```
13      cin >> a >> b;
14      n = a.size(); m = b.size();
15      a = " " + a; b = " " + b;
16
17      // 初始化
18      for(int i = 1; i <= n; i++) f[i][0] = i;
19      for(int j = 1; j <= m; j++) f[0][j] = j;
20
21      for(int i = 1; i <= n; i++)
22          for(int j = 1; j <= m; j++)
23          {
24              if(a[i] == b[j]) f[i][j] = f[i - 1][j - 1];
25              else f[i][j] = min(min(f[i - 1][j], f[i - 1][j - 1]), f[i][j -
    1]) + 1;
26          }
27
28      cout << f[n][m] << endl;
29
30      return 0;
31  }
```

3. 背包问题

背包问题是动态规划中最经典的问题，很多题目或多或少都有背包问题的影子。它的基本形式是：给定一组物品，每个物品有体积和价值，在不超过背包容量的情况下，选择物品使得总价值最大。

背包问题有多种变体，主要包括：

1. **01 背包问题**：每种物品只能选或不选（选 0 次或 1 次）。
2. **完全背包问题**：每种物品可以选择无限次。
3. **多重背包问题**：每种物品有数量限制。
4. **分组背包问题**：物品被分为若干组，每组只能选一个物品。
5. **混合背包**：以上四种背包问题混在一起。
6. **多维费用的背包问题**：限定条件不止有体积，还会有其他因素（比如重量）。

除了经典的总价值最大问题，还会有：

1. **方案总数。**
2. **最优方案。**
3. **方案可行性。**

4. **输出具体方案。**

因此，背包问题种类非常繁多，题型非常丰富。但是，尽管背包有很多变形，都是从 01 背包问题演化过来的。所以，一定要把 01 背包问题学好。

3.1 01 背包

3.1.1 01 背包

> **题目来源：** 牛客网
>
> **题目链接：** 【模板】01背包
>
> **难度系数：** ★★

【题目描述】

> 你有一个背包，最多能容纳的体积是 V 。
>
> 现在有 n 个物品，第 i 个物品的体积为 v_i ,价值为 w_i 。
>
> 1. 求这个背包至多能装多大价值的物品?
> 2. 若背包恰好装满，求至多能装多大价值的物品?

【输入描述】

> 第一行两个整数 n 和 V ，表示物品个数和背包体积。
>
> 接下来 n 行，每行两个数 v_i 和 w_i ，表示第 i 个物品的体积和价值。
>
> $1 \leq n, V, v_i, w_i \leq 1000$

【输出描述】

> 输出有两行，第一行输出第一问的答案，第二行输出第二问的答案，如果无解请输出 0 。

【示例一】

> 输入：
>
> ```
> 3 5
> 2 10
> 4 5
> 1 4
> ```
>
> 输出：
>
> ```
> 14
> ```

> 9

【示例二】

> 输入：
>
> 3 8
>
> 12 6
>
> 11 8
>
> 6 8
>
> 输出：
>
> 8
>
> 0

【解法】

我们先解决第一问：

1. 状态表示：

   $dp[i][j]$ 表示：从前 i 个物品中挑选，总体积「不超过」 j ，所有的选法中，能挑选出来的最大价值。

   那么 $dp[n][v]$ 就是我们要的结果。

2. 状态转移方程：

   线性 dp 状态转移方程分析方式，一般都是根据「最后一步」的状况，来分情况讨论：

   a. 不选第 i 个物品：相当于就是去前 $i-1$ 个物品中挑选，并且总体积不超过 j 。此时 $dp[i][j]=dp[i-1][j]$ ；

   b. 选择第 i 个物品：那么我就只能去前 $i-1$ 个物品中，挑选总体积不超过 $j-v[i]$ 的物品。此时 $dp[i][j]=dp[i-1][j-v[i]]+w[i]$ 。但是这种状态不一定存在，因此需要特判一下。

   综上，状态转移方程为：$dp[i][j]=max(dp[i-1][j],dp[i-1][j-v[i]]+w[i])$ 。

3. 初始化：

   直接填表，第一行的 0 不影响结果。

4. 填表顺序：

   根据「状态转移方程」，我们仅需「从上往下」填表即可。

接下来解决第二问：

第二问仅需修改一下初始化以及最终结果即可。

1. 初始化：

   因为有可能凑不齐 j 体积的物品，因此我们把不合法的状态设置为负无穷。这样在取最大值的时候，就不会考虑到这个位置的值。负无穷一般设置为 $-0x3f3f3f3f$ 即可。

   然后把 $dp[0][0]=0$ 修改成 0 ，因为这是一个合法的状态，最大价值是 0 ，也让后续填表是正确的。

2. 返回值：

   在最后拿结果的时候，也要判断一下最后一个位置是不是小于 0 ，因为有可能凑不齐。

   不能判断是否等于 $-0x3f3f3f3f$ ，因为这个位置的值会被更新，只不过之前的值太小，导致更新后还是小于 0 的。

【参考代码】（后面有空间优化的版本）

```cpp
// 原始版本
#include <iostream>
#include <cstring>

using namespace std;

const int N = 1010;

int n, m;
int v[N], w[N];
int f[N][N];

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> v[i] >> w[i];

    // 第一问
    for(int i = 1; i <= n; i++)
    {
        for(int j = 0; j <= m; j++)
        {
            f[i][j] = f[i - 1][j];
            if(j >= v[i])
            {
```

```cpp
                f[i][j] = max(f[i][j], f[i - 1][j - v[i]] + w[i]);
            }
        }
    }

    cout << f[n][m] << endl;

    // 第二问
    // 初始化
    memset(f, -0x3f, sizeof f);
    f[0][0] = 0;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 0; j <= m; j++)
        {
            f[i][j] = f[i - 1][j];
            if(j >= v[i])
            {
                f[i][j] = max(f[i][j], f[i - 1][j - v[i]] + w[i]);
            }
        }
    }

    if(f[n][m] < 0) cout << 0 << endl;
    else cout << f[n][m] << endl;

    return 0;
}

// 空间优化的 01背包
#include <iostream>
#include <cstring>

using namespace std;

const int N = 1010;

int n, m;
int v[N], w[N];
int f[N];

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> v[i] >> w[i];

    // 第一问
```

```cpp
    for(int i = 1; i <= n; i++)
    {
        for(int j = m; j >= v[i]; j--) // 修改遍历顺序
        {
            f[j] = max(f[j], f[j - v[i]] + w[i]);
        }
    }

    cout << f[m] << endl;

    // 第二问
    // 初始化
    memset(f, -0x3f, sizeof f);
    f[0] = 0;
    for(int i = 1; i <= n; i++)
    {
        for(int j = m; j >= v[i]; j--) // 修改遍历顺序
        {
            f[j] = max(f[j], f[j - v[i]] + w[i]);
        }
    }

    if(f[m] < 0) cout << 0 << endl;
    else cout << f[m] << endl;

    return 0;
}
```

3.1.2 采药

> **题目来源：** 洛谷
>
> **题目链接：** P1048 [NOIP2005 普及组] 采药
>
> **难度系数：** ★

【题目描述】

> 辰辰是个天资聪颖的孩子，他的梦想是成为世界上最伟大的医师。为此，他想拜附近最有威望的医师为师。医师为了判断他的资质，给他出了一个难题。医师把他带到一个到处都是草药的山洞里对他说：“孩子，这个山洞里有一些不同的草药，采每一株都需要一些时间，每一株也有它自身的价值。我会给你一段时间，在这段时间里，你可以采到一些草药。如果你是一个聪明的孩子，你应该可以让采到的草药的总价值最大。”
>
> 如果你是辰辰，你能完成这个任务吗？

【输入描述】

> 第一行有 2 个整数 $T(1 \leq T \leq 1000)$ 和 $M(1 \leq M \leq 100)$，用一个空格隔开，T 代表总共能够用来采药的时间，M 代表山洞里的草药的数目。
>
> 接下来的 M 行每行包括两个在 1 到 100 之间（包括 1 和 100）的整数，分别表示采摘某株草药的时间和这株草药的价值。

【输出描述】

> 输出在规定的时间内可以采到的草药的最大总价值。

【示例一】

> 输入：
>
> 70 3
>
> 71 100
>
> 69 1
>
> 1 2
>
> 输出：
>
> 3

【解法】

基本 01 背包问题，将时间看成体积，就是标准的不放满的 01 背包问题。不再赘述~

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1010;

int n, m;
int t[N], w[N];
int f[N];

int main()
{
    cin >> m >> n;
    for(int i = 1; i <= n; i++) cin >> t[i] >> w[i];

    for(int i = 1; i <= n; i++)
        for(int j = m; j >= t[i]; j--) // 修改遍历顺序
        {
            f[j] = max(f[j], f[j - t[i]] + w[i]);
```

```
            }

        cout << f[m] << endl;

        return 0;
    }
```

3.1.3 小 A 点菜

> **题目来源：** 洛谷
>
> **题目链接：** P1164 小A点菜
>
> **难度系数：** ★★

【题目描述】

> uim 神犇拿到了 uoi 的 ra（镭牌）后，立刻拉着基友小 A 到了一家……餐馆，很低端的那种。
>
> uim 指着墙上的价目表（太低级了没有菜单），说：“随便点”。
>
> 不过 uim 由于买了一些书，口袋里只剩 M 元 $(M \leq 10000)$。
>
> 餐馆虽低端，但是菜品种类不少，有 N 种 $(N \leq 100)$，第 i 种卖 a_i 元 $(a_i \leq 1000)$。由于是很低端的餐馆，所以每种菜只有一份。
>
> 小 A 奉行“不把钱吃光不罢休”的原则，所以他点单一定刚好把 uim 身上所有钱花完。他想知道有多少种点菜方法。
>
> 由于小 A 肚子太饿，所以最多只能等待 1 秒。

【输入描述】

> 第一行是两个数字，表示 N 和 M。
>
> 第二行起 N 个正数 a_i（可以有相同的数字，每个数字均在 1000 以内）。

【输出描述】

> 一个正整数，表示点菜方案数，保证答案的范围在 int 之内。

【示例一】

> 输入：
>
> ```
> 4 4
> 1 1 2 2
> ```
>
> 输出：
>
> ```
> 3
> ```

【解法】

背包问题求方案数，稍微修改一个状态表示，然后根据具体问题分析状态转移方程和初始化即可。

1. 状态表示：

   $dp[i][j]$ 表示：从前 i 个菜中挑选，总价钱恰好等于 j ，此时的总方案数。

2. 状态转移方程：

   针对 $a[i]$ 选或者不选，分两种情况讨论：

   a. 如果不选 $a[i]$ ：相当于去前 $i-1$ 个菜中挑选，总价钱恰好为 j 的方案数，此时的方案数就是 $dp[i-1][j]$ ；

   b. 如果选 $a[i]$ ：那么应该去前 $i-1$ 个菜中挑选，总价值恰好为 $j-a[i]$ ，此时的方案数就是 $dp[i-1][j-a[i]]$ ；

   因为我们要的是总方案数，于是 $dp[i][j]=dp[i-1][j]+dp[i-1][j-a[i]]$ 。

   注意第二个状态可能不存在，要注意判断一下 $j \geq a[i]$ 。

3. 初始化：

   $dp[0][0]=1$ ，如果没有物品，想凑成总体积为 0 是可行的，啥也不选就是一种方案。当然，这个状态也是为了让后面的值是正确的。

   其余位置的值是 0 就不影响填表的正确性。

4. 填表顺序：

   从上往下每一行，每一行从左往右。

   空间优化版本：每一行从右往左。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 110, M = 10010;

int n, m;
int a[N];
int f[N][M];

int main()
```

```cpp
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> a[i];

    f[0][0] = 1;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 0; j <= m; j++)
        {
            f[i][j] = f[i - 1][j];
            if(j >= a[i]) f[i][j] += f[i - 1][j - a[i]];
        }
    }

    cout << f[n][m] << endl;

    return 0;
}
```

3.1.4 Cow Frisbee Team

题目来源： 洛谷

题目链接： P2946 [USACO09MAR] Cow Frisbee Team S

难度系数： ★★★

【题目描述】

老唐最近迷上了飞盘，约翰想和他一起玩，于是打算从他家的 N 头奶牛中选出一支队伍。

每只奶牛的能力为整数，第 i 头奶牛的能力为 R_i。飞盘队的队员数量不能少于 1、大于 N。一支队伍的总能力就是所有队员能力的总和。

约翰比较迷信，他的幸运数字是 F，所以他要求队伍的总能力必须是 F 的倍数。请帮他算一下，符合这个要求的队伍组合有多少？由于这个数字很大，只要输出答案对 10^8 取模的值。

【输入描述】

第一行：两个用空格分开的整数：N 和 F。

第二行到 $N+1$ 行：第 $i+1$ 行有一个整数 R_i，表示第 i 头奶牛的能力。

对于 100% 的数据，$1 \le N \le 2000$，$1 \le F \le 1000$，$1 \le R_i \le 10^5$。

【输出描述】

第一行：单个整数，表示方案数对 10^8 取模的值。

【示例一】

输入：

```
4 5
1
2
8
2
```

输出：

```
3
```

【解法】

01 背包问题变形。

1. 状态表示：

   $dp[i][j]$ 表示：从前 i 头奶牛中挑选，总和模 f 之后为 j 时，一共有多少种组合。

   那么 $dp[n][0]-1$ 就是最终结果。（因为动态规划会把全都不选这种情况也考虑进去，所以要剪掉）

2. 状态转移方程：

   对于第 i 头奶牛选或者不选，可以分为两种情况讨论：

   a. 如果不选 $a[i]$：此时的总方案数就是去 $[1,i-1]$ 里面凑余数正好是 j，也就是 $dp[i-1][j]$；

   b. 如果选 $a[i]$：此时已经有一个余数为 $a[i]\%f$，只用再去前面凑一个 $j-a[i]\%f$ 即可。但是直接减可能会减出来一个负数，我们要把它补正，最终凑的数为 $(j-a[i]\%f+f)\%f$。那么总方案数就是 $dp[i-1][(j-a[i]\%f+f)\%f]$。

   因为要的总方案数，所以状态转移方程就是上面两种情况的总和。

3. 初始化：

   $dp[0][0]=1$：什么也不选的时候，总和是 0，余数也是 0，属于一种方案，也是为了后续填表是正确的。

4. 填表顺序：

   从上往下每一行，每一行从左往右。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 2010, M = 1010, MOD = 1e8;

int n, m;
int a[N];
int f[N][M];

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> a[i];

    f[0][0] = 1;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            f[i][j] = (f[i - 1][j] + f[i - 1][((j - a[i] % m) % m + m) % m]) % MOD;
        }
    }

    cout << f[n][0] - 1 << endl;

    return 0;
}
```

3.2 完全背包

3.2.1 完全背包

> **题目来源：** 牛客网
>
> **题目链接：** 【模板】完全背包
>
> **难度系数：** ★★

【题目描述】

> 你有一个背包，最多能容纳的体积是 V 。

> 现在有 n 种物品，每种物品有任意多个，第 i 种物品的体积为 v_i ,价值为 w_i 。
>
> 1. 求这个背包至多能装多大价值的物品?
> 2. 若背包恰好装满，求至多能装多大价值的物品?

【输入描述】

> 第一行两个整数 n 和 V ，表示物品个数和背包体积。
>
> 接下来 n 行，每行两个数 v_i 和 w_i ，表示第 i 种物品的体积和价值。
>
> $1 \leq n, V \leq 1000$

【输出描述】

> 输出有两行，第一行输出第一问的答案，第二行输出第二问的答案，如果无解请输出 0 。

【示例一】

> 输入:
>
> 2 6
>
> 5 10
>
> 3 1
>
> 输出:
>
> 10
>
> 2

【解法】

先解决第一问:

1. 状态表示:

   $dp[i][j]$ 表示：从前 i 个物品中挑选，总体积不超过 j ，所有的选法中，能挑选出来的最大价值。（这里是和 01 背包一样哒）

   那我们的最终结果就是 $dp[n][V]$ 。

2. 状态转移方程:

   线性 dp 状态转移方程分析方式，一般都是根据最后一步的状况，来分情况讨论。但是最后一个物品能选很多个，因此我们的需要分很多情况:

   a. 选 0 个第 i 个物品：此时相当于就是去前 $i-1$ 个物品中挑选，总体积不超过 j 。此时最大价值为 $dp[i-1][j]$;

   b. 选 1 个第 i 个物品：此时相当于就是去前 $i-1$ 个物品中挑选，总体积不超过 $j-v[i]$ 。因为挑选了一个 i 物品，此时最大价值为 $dp[i-1][j-v[i]]+w[i]$;

c. 选 2 个第 i 个物品：此时相当于就是去前 $i-1$ 个物品中挑选，总体积不超过 $j-2\times v[i]$ 。因为挑选了两个 i 物品，此时最大价值为 $dp[i-1][j-2\times v[i]]+2\times w[i]$ ；

d. .....

综上，状态转移方程为：

$$dp[i][j]=max(dp[i-1][j],dp[i-1][j-v[i]]+w[i],dp[i-1][j-2\times v[i]]+2\times w[i]...$$

当计算一个状态的时候，需要一个循环才能搞定的时候，就要想到去优化。优化的方向就是用一个或者两个状态来表示这一堆的状态，通常就是用数学的方式做一下等价替换。

观察发现第二维是有规律的变化的，因此去看看 $dp[i][j-v[i]]$ 这个状态：

$dp[i][j-v[i]]=max(dp[i-1][j-v[i]],dp[i-1][j-2\times v[i]]+w[i],dp[i-1][j-3\times v[i]]+2\times w[i]...)$

我们发现，把 $dp[i][j-v[i]]$ 加上 $w[i]$ 正好和 $dp[i][j]$ 中除了第一项以外的全部一致，因们可以修改状态转移方程为： $dp[i][j]=max(dp[i-1][j],dp[i][j-v[i]]+w[i])$

3. 初始化：

我们多加一行，方便我们的初始化，此时仅需将第一行初始化为 0 即可。因为什么也不选，也能满足体积不小于 j 的情况，此时的价值为 0 。

4. 填表顺序：

根据状态转移方程，我们仅需从上往下填表即可。

接下来解决第二问：

第二问仅需修改一下初始化以及最终结果即可。

1. 初始化：

因为有可能凑不齐 j 体积的物品，因此我们把不合法的状态设置为负无穷。这样在取最大值的时候，就不会考虑到这个位置的值。负无穷一般设置为 $-0x3f3f3f3f$ 即可。

然后把 $dp[0][0]=0$ 修改成 0 ，因为这是一个合法的状态，最大价值是 0 ，也让后续填表是正确的。

2. 返回值：

在最后拿结果的时候，也要判断一下最后一个位置是不是小于 0 ，因为有可能凑不齐。

不能判断是否等于 $-0x3f3f3f3f$，因为这个位置的值会被更新，只不过之前的值太小，导致更新后还是小于 0 的。

【参考代码】（后续还有空间优化的版本）

```cpp
// 原始版本
#include <iostream>
#include <cstring>

using namespace std;

const int N = 1010;

int n, m;
int v[N], w[N];
int f[N][N];

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> v[i] >> w[i];

    // 第一问
    for(int i = 1; i <= n; i++)
    {
        for(int j = 0; j <= m; j++)
        {
            f[i][j] = f[i - 1][j];
            if(j >= v[i]) f[i][j] = max(f[i][j], f[i][j - v[i]] + w[i]);
        }
    }

    cout << f[n][m] << endl;

    // 第二问
    memset(f, -0x3f, sizeof f);
    f[0][0] = 0;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 0; j <= m; j++)
        {
            f[i][j] = f[i - 1][j];
            if(j >= v[i]) f[i][j] = max(f[i][j], f[i][j - v[i]] + w[i]);
        }
    }
```

```
41
42      if(f[n][m] < 0) cout << 0 << endl;
43      else cout << f[n][m] << endl;
44
45      return 0;
46  }
47
48  // 空间优化的版本
49  #include <iostream>
50  #include <cstring>
51
52  using namespace std;
53
54  const int N = 1010;
55
56  int n, m;
57  int v[N], w[N];
58  int f[N];
59
60  int main()
61  {
62      cin >> n >> m;
63      for(int i = 1; i <= n; i++) cin >> v[i] >> w[i];
64
65      // 第一问
66      for(int i = 1; i <= n; i++)
67      {
68          for(int j = v[i]; j <= m; j++) // 从小到大循环
69          {
70              f[j] = max(f[j], f[j - v[i]] + w[i]);
71          }
72      }
73
74      cout << f[m] << endl;
75
76      // 第二问
77      memset(f, -0x3f, sizeof f);
78      f[0] = 0;
79      for(int i = 1; i <= n; i++)
80      {
81          for(int j = v[i]; j <= m; j++) // 从小到大循环
82          {
83              f[j] = max(f[j], f[j - v[i]] + w[i]);
84          }
85      }
86
87      if(f[m] < 0) cout << 0 << endl;
```

```
88        else cout << f[m] << endl;
89
90        return 0;
91    }
```

3.2.2 疯狂的采药

题目来源： 洛谷

题目链接： P1616 疯狂的采药

难度系数： ★

【题目描述】

LiYuxiang 是个天资聪颖的孩子，他的梦想是成为世界上最伟大的医师。为此，他想拜附近最有威望的医师为师。医师为了判断他的资质，给他出了一个难题。医师把他带到一个到处都是草药的山洞里对他说：“孩子，这个山洞里有一些不同种类的草药，采每一种都需要一些时间，每一种也有它自身的价值。我会给你一段时间，在这段时间里，你可以采到一些草药。如果你是一个聪明的孩子，你应该可以让采到的草药的总价值最大。”

如果你是 LiYuxiang，你能完成这个任务吗?

此题和原题的不同点：

1. 每种草药可以无限制地疯狂采摘。
2. 药的种类眼花缭乱，采药时间好长好长啊！师傅等得菊花都谢了！

【输入描述】

输入第一行有两个整数，分别代表总共能够用来采药的时间 t 和代表山洞里的草药的数目 m。

第 2 到第 $(m+1)$ 行，每行两个整数，第 $(i+1)$ 行的整数 a_i, b_i 分别表示采摘第 i 种草药的时间和该草药的价值。

对于 100% 的数据，保证 $1 \le m \le 10^4, 1 \le t \le 10^7$，且 $1 \le m \times t \le 10^7, 1 \le a_i, b_i \le 10^4$。

【输出描述】

输出一行，这一行只包含一个整数，表示在规定的时间内，可以采到的草药的最大总价值。

【示例一】

输入：

70 3

71 100

69 1

```
1 2
```

输出：

```
140
```

【解法】

完全背包模版题。

1. 状态表示：

   $dp[i][j]$ 表示：从前 i 个药材中挑选，总时间不超过 j ，此时能采摘到的最大价值。

   那么 $dp[n][m]$ 就是结果。

2. 状态转移方程：

   对于 i 位置的药材，可以选择采 $0, 1, 2, 3...$ 个：

   a. 选 0 个：最大价值为 $dp[i-1][j]$ ；

   b. 选 1 个：最大价值为 $dp[i-1][j-t[i]]+v[i]$ ；

   c. 选 2 个：最大价值为 $dp[i-1][j-2\times t[i]]+2\times v[i]$ ；

   d. ...

   由于要的是最大价值，应该是上述所有情况的最大值。其中第二个往后的状态可以用 $dp[i][j-t[i]]+v[i]$ 替代，因此状态转移方程为 $dp[i][j]=max(dp[i-1][j], dp[i][j-t[i]]+v[i])$ 。

3. 初始化：

   全部初始化 0 ，不影响后续填表的正确性。

4. 填表顺序：

   从上往下每一行，每一行从左往右。

   空间优化版本也是如此。

【参考代码】

```
1  #include <iostream>
2
3  using namespace std;
4
5  typedef long long LL;
```

```cpp

const int N = 1e4 + 10, M = 1e7 + 10;

int n, m;
int t[N], w[N];
LL f[M];

int main()
{
    cin >> m >> n;
    for(int i = 1; i <= n; i++) cin >> t[i] >> w[i];

    for(int i = 1; i <= n; i++)
    {
        for(int j = t[i]; j <= m; j++)
        {
            f[j] = max(f[j], f[j - t[i]] + w[i]);
        }
    }

    cout << f[m] << endl;

    return 0;
}
```

3.2.3 Buying Hay

题目来源： 洛谷

题目链接： P2918 [USACO08NOV] Buying Hay S

难度系数： ★★

【题目描述】

约翰的干草库存已经告罄，他打算为奶牛们采购 $H(1 \leq H \leq 50000)$ 磅干草。

他知道 $N(1 \leq N \leq 100)$ 个干草公司，现在用 1 到 N 给它们编号。第 i 公司卖的干草包重量为 $P_i(1 \leq P_i \leq 5,000)$ 磅，需要的开销为 $C_i(1 \leq C_i \leq 5,000)$ 美元。每个干草公司的货源都十分充足， 可以卖出无限多的干草包。

帮助约翰找到最小的开销来满足需要，即采购到至少 H 磅干草。

【输入描述】

第一行：两个整数，分别是 N 和 H

接下来 N 行：每行两个整数，分别表示 P_i 和 C_i

【输出描述】

> 一个整数，表示约翰至少采购 H 磅干草所用的最小开销。

【示例一】

> 输入：
>
> 2 15
>
> 3 2
>
> 5 3
>
> 输出：
>
> 9

【解法】

完全背包简单变形。

1. 状态表示：

   $dp[i][j]$ 表示：从前 i 个干草公司中挑选，总重量至少为 j 磅，此时的最小开销。

   注意注意注意！这是我们遇到的第三类限制情况。

   之前的限制条件为不超过 j ，或者是恰好等于 j 。这道题的限定条件是至少为 j ，也就是说可以超过 j 。这会对我们分析状态转移方程的时候造成影响。

   根据状态表示， $dp[n][m]$ 就是结果。

2. 状态转移方程：

   对于 i 位置的公司，可以选择买 $0, 1, 2, 3...$ 个：

   a. 选 0 个：开销为 $dp[i-1][j]$ ；

   b. 选 1 个：开销为 $dp[i-1][j-p[i]]+c[i]$ 。问题来了，状态表示里面是至少为 j ，也就是说 $j-p[i]$ 小于 0 也是合法的。因为公司提供了 $p[i]$ 的重量，大于 j ，是符合要求的。但是 dp 表的下标不能是负数，处理这种情况的方式就是对 $j-p[i]$ 与 0 取一个最大值 $max(j-p[i], 0)$ 。当重量很大的时，只用去前面凑重量为 0 的就足够了，这样就符合我们的状态表示了。因此，最终开销为 $dp[i-1][max(0, j-p[i])]+c[i]$

   c. 选 2 个：开销为 $dp[i-1][max(0, j-2\times p[i])]+2\times c[i]$ ；

   d. ...

   由于要的是最小开销，应该是上述所有情况的最小值。

   其中第二个往后的状态可以用 $dp[i][max(0, j-p[i])]+c[i]$ 替代，因此状态转移方程为 $dp[i][j]=max(dp[i-1][j], dp[i][max(0, j-p[i])]+c[i])$ 。

3. 初始化：

   全部初始化正无穷大 $0x3f3f3f3f$，然后 $dp[0][0]=0$，不影响后续填表的正确性。

4. 填表顺序：

   从上往下每一行，每一行从左往右。

   空间优化版本也是如此。

【参考代码】

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 110, M = 50010;

int n, m;
int p[N], c[N];
int f[M];

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> p[i] >> c[i];

    memset(f, 0x3f, sizeof f);
    f[0] = 0;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 0; j <= m; j++)
        {
            f[j] = min(f[j], f[max(0, j - p[i])] + c[i]);
        }
    }

    cout << f[m] << endl;

    return 0;
}
```

3.2.4 纪念品

题目来源： 洛谷

题目链接： P5662 [CSP-J2019] 纪念品

难度系数： ★★★

【题目描述】

小伟突然获得一种超能力，他知道未来 T 天 N 种纪念品每天的价格。某个纪念品的价格是指购买一个该纪念品所需的金币数量，以及卖出一个该纪念品换回的金币数量。

每天，小伟可以进行以下两种交易无限次：

1. 任选一个纪念品，若手上有足够金币，以当日价格购买该纪念品；
2. 卖出持有的任意一个纪念品，以当日价格换回金币。

每天卖出纪念品换回的金币可以立即用于购买纪念品，当日购买的纪念品也可以当日卖出换回金币。当然，一直持有纪念品也是可以的。

T 天之后，小伟的超能力消失。因此他一定会在第 T 天卖出所有纪念品换回金币。

小伟现在有 M 枚金币，他想要在超能力消失后拥有尽可能多的金币。

【输入描述】

第一行包含三个正整数 T, N, M，相邻两数之间以一个空格分开，分别代表未来天数 T，纪念品数量 N，小伟现在拥有的金币数量 M。

接下来 T 行，每行包含 N 个正整数，相邻两数之间以一个空格分隔。

第 i 行的 N 个正整数分别为 $P_{i,1}, P_{i,2}, ..., P_{i,N}$，其中 $P_{i,j}$ 表示第 i 天第 j 种纪念品的价格。

【输出描述】

输出仅一行，包含一个正整数，表示小伟在超能力消失后最多能拥有的金币数量。

【示例一】

输入：

```
6 1 100
50
20
25
20
25
50
```

输出：

> 305

【示例二】

> 输入：
>
> 3 3 100
>
> 10 20 15
>
> 15 17 13
>
> 15 25 16
>
> 输出：
>
> 217

【解法】

总策略：贪心。从前往后，一天一天的考虑如何最大金币。

因为纪念品可以在当天买，当天卖。因此所有的交易情况，就可以转换成"某天买隔天卖"的情况。那么，我们就可以贪心的将每一天能拿到的最大利润全都拿到手：

- 从第一天开始，把第一天的金币看成限制条件，第二天的金币看成价值，求出：在不超过 m 的情况下，能获得的最大价值 m_1；
- 然后时间来到第二天，把这一天的金币看成限制条件，第三天的金币看成价值，求出：在不超 m_1 的情况下，能获得的最大价值 m_2；
- 以此类推，直到把第 $t-1$ 天的情况计算出来，能获得最大价值就是结果。

接下来就处理，拿到第 i 行以及 $i+1$ 行数据，在最大金币数量为 m 的前提下，获得的最大利润是多少？

- 因为每一个纪念品都可以无限次购买；
- 把前一行看成限制，后一行减去前一行的值看成价值，就变成了标准的完全背包问题。

那我们的解决方法就是一行一行的跑完全背包，跑一行拿到最大价值，然后放到下一行继续跑，直到跑完倒数第二行。

完全背包的逻辑：

1. 状态表示：

   $dp[i][j]$ 表示从前 i 个纪念品中挑选，总花费不超过 j 的情况下，最大的利润。

   那么，$dp[n][m]+m$ 就是能得到的最大金币数量。

2. 状态转移方程：

根据最后一个纪念品选的数量，分成如下情况：

a. 如果选 0 个：能获得的最大金币数量为 $dp[i-1][j]$；

b. 如果选 1 个：能获得的最大金币数量为 $dp[i-1][j-w[i]]+v[i]-w[i]$；

c. 如果选 2 个：能获得的最大金币数量为 $dp[i-1][j-2\times w[i]]+2\times(v[i]-w[i])$；

d. ...

其中除了第一个状态外的所有状态都可以用 $dp[i][j-w[i]]+v[i]-w[i]$ 来表示，又因为要的是最大值，所以状态转移方程就是所有情况的最大值。

3. 初始化

全为 0 即可。

【参考代码】

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 110, M = 1e4 + 10;

int t, n, m;
int p[N][N];
int f[M];

// 完全背包
int solve(int v[], int w[], int m)
{
    // 清空数据
    memset(f, 0, sizeof f);
    for(int i = 1; i <= n; i++)
    {
        for(int j = v[i]; j <= m; j++)
        {
            f[j] = max(f[j], f[j - v[i]] + w[i] - v[i]);
        }
    }

    return m + f[m];
}
```

```cpp

int main()
{
    cin >> t >> n >> m;
    for(int i = 1; i <= t; i++)
        for(int j = 1; j <= n; j++)
            cin >> p[i][j];

    // 贪心
    for(int i = 1; i < t; i++)
    {
        m = solve(p[i], p[i + 1], m);
    }

    cout << m << endl;

    return 0;
}
```

3.3 多重背包

3.3.1 多重背包

> **题目来源：** 牛客网
>
> **题目链接：** 多重背包
>
> **难度系数：** ★★

【题目描述】

> 有 n 种物品，第 i 种物品有 x_i 个，每一个物品重量为 w_i ，价值为 v_i ，现有一个承重能力为 T 的背包，在不超过承重能力的情况下，背包种最多能装多少价值的物品。

【输入描述】

> 第一行输入两个整数 $n, T(1 \leq n, T \leq 100)$ ，代表物品种数和背包承重能力
>
> 接下来 n 行，每行三个整数 $x_i, w_i, v_i(1 \leq x_i, w_i \leq 20, 1 \leq v_i \leq 200)$ 描述一个物品，分别代表物品的个数、物品的重量、物品的价值。

【输出描述】

> 输出一行一个整数，表示在不超过承重能力的情况下，背包物品的最大价值。

【示例一】

> 输入：

> 2 8
>
> 4 2 100
>
> 2 4 100
>
> 输出：
>
> 400

【解法】

多重背包问题有两种解法：

1. 按照背包问题的常规分析方式，仿照完全背包，第三维枚举使用的个数；
2. 利用二进制可以表示一定范围内整数的性质，转化成 01 背包问题。

小建议：并不是所有的多重背包问题都能用二进制优化，而且优化版本的代码很长。因此，如果时间复杂度允许的情况下，能不优化就不优化~~~

解法一：常规分析

1. 状态表示：

   $dp[i][j]$ 表示：从前 i 个物品中挑选，总重量不超过 j 的情况下，最大的价值。

   $dp[n][m]$ 就是最终结果。

2. 状态转移方程：

   根据第 i 个物品选的个数，可以分 $x[i]+1$ 种情况：

   a. 选 0 个：价值为 $dp[i-1][j]$；

   b. 选 1 个：价值为 $dp[i-1][j-w[i]]+v[i]$；

   c. 选 2 个：价值为 $dp[i-1][j-2\times w[i]]+2\times v[i]$；

   d. ...

   e. 选 $x[i]$ 个：价值为 $dp[i-1][j-x[i]\times w[i]]+x[i]\times v[i]$。

   因为要的是最大价值，所以 $dp[i][j]$ 等于上述所有情况的最大值。但是要注意 $j-k*w[i]$ 要大于等于 0，并且不能按照完全背包的方式优化。（大家可以试着写一写，是不能由之前的状态表示的）

3. 初始化：

   全部为 0 就不影响最终结果。

4. 填表顺序：

   从上往下每一行，每一行从左往右。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 110;

int n, m;
int x[N], w[N], v[N];
int f[N];

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> x[i] >> w[i] >> v[i];

    for(int i = 1; i <= n; i++)
        for(int j = m; j >= 0; j--)
            for(int k = 0; k <= x[i] && k * w[i] <= j; k++)
                f[j] = max(f[j], f[j - k * w[i]] + k * v[i]);

    cout << f[m] << endl;

    return 0;
}
```

解法二：转化成 01 背包问题

优化方式：用二进制将 $x[i]$ 个物品分组。

连续的二进制数有一个性质，就是 $2^0 \sim 2^k$ 能够表示区间 $[1, 2^{k+1} - 1]$ 里面所有的整数。比如：

- $1, 2, 4, 8$ 可以表示 $[1, 15]$ 内所有的整数。具体原因可以参考整数的二进制表示，$1, 2, 4, 8$ 正好对应二进制表示中每一位的权值，所以排列组合起来就可以表示 $[1, 15]$ 内所有的整数。
- 同理 $1, 2, 4$ 就可以表示 $[1, 7]$ 内所有的整数。

根据这样一个性质，我们就可以把 $x[i]$ 拆成一些二进制数再加上多出来的数，这样的一组数就可以表示 $[1, x[i]]$ 内所有的整数，问题就变成了 01 背包。

比如 $x[i]=9$，$w[i]=2, v[i]=3$：

- $9=1+2+4+2$；
- 分成 4 组，每组的重量和价值分别为 $(2,3)$、$(4,6)$、$(8,12)$、$(4,6)$ 。

```cpp
#include <iostream>

using namespace std;

const int N = 110 * 5;

int n, m;
int w[N], v[N], pos;
int f[N];

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
    {
        int x, y, z; cin >> x >> y >> z;
        // 按照二进制拆分
        int t = 1;
        while(x >= t)
        {
            pos++;
            w[pos] = t * y;
            v[pos] = t * z;
            x -= t;
            t *= 2;
        }
        if(x) // 处理剩余
        {
            pos++;
            w[pos] = x * y;
            v[pos] = x * z;
        }
    }

    // 针对拆分之后的物品，做一次 01背包即可
    for(int i = 1; i <= pos; i++)
        for(int j = m; j >= w[i]; j--)
            f[j] = max(f[j], f[j - w[i]] + v[i]);

```

```
40        cout << f[m] << endl;
41
42        return 0;
43    }
```

3.3.2 摆花

题目来源： 洛谷

题目链接： P1077 [NOIP2012 普及组] 摆花

难度系数： ★★

【题目描述】

小明的花店新开张，为了吸引顾客，他想在花店的门口摆上一排花，共 m 盆。通过调查顾客的喜好，小明列出了顾客最喜欢的 n 种花，从 1 到 n 标号。为了在门口展出更多种花，规定第 i 种花不能超过 a_i 盆，摆花时同一种花放在一起，且不同种类的花需按标号的从小到大的顺序依次摆列。

试编程计算，一共有多少种不同的摆花方案。

【输入描述】

第一行包含两个正整数 n 和 m，中间用一个空格隔开。

第二行有 n 个整数，每两个整数之间用一个空格隔开，依次表示 $a_1, a_2, \cdots, a_n$。

对于 100% 数据，有 $0 < n \leq 100, 0 < m \leq 100, 0 \leq a_i \leq 100$。

【输出描述】

一个整数，表示有多少种方案。注意：因为方案数可能很多，请输出方案数对 $10^6 + 7$ 取模的结果。

【示例一】

输入：

```
2 4
3 2
```

输出：

```
2
```

【解法】

题意：每一种花可以选 $[0, a[i]]$ 个，在总数恰好等于 m 时的总方案数。

正好是多重背包求方案数的模型，我们可以用多重背包的思考方式来解决这道题。

1. 状态表示：

   $dp[i][j]$ 表示：从前 i 个花中挑选，正好摆放 j 个花盆时的方案数。

2. 状态转移方程：

   根据第 i 种花选的个数 $k(0 \le k \le min(j, a[i]))$ 分情况讨论：

   - 如果当前花选了 k 盆，之前的花要去凑够 $j-k$ 盆，总的方案数就是 $dp[i-1][j-k]$；
   - 因为要的是总方案数，所以最终结果应该是 k 的变化过程中的状态的总和。

   $dp[i][j] = dp[i][j] + dp[i-1][j-k]$

3. 初始化：

   $dp[0][0] = 1$，相当于起始状态，为了让后续的填表有意义，不然全都是 0。

4. 填表顺序：

   从上往下每一行，每一行从左往右。

这道题就不能用二进制优化，因为这道题的背包，限定条件和价值是一一对应的，并且求的是方案数。如果用二进制优化会统计多余的情况，比如：

- 有两个物品，个数分别是 $3, 2$，要凑成总和为 4。
- 拆分之后为 $(1, 2)$、$(1, 1)$，跑一遍 01 背包之后，结果是 3，但是实际情况应该是 2。
- 原因是 $1, 2, 1$ 被统计了 2 次。但是在实际情况里，第一个物品全选，第二个物品选 1 个，只属于 1 种情况，而 01 背包的逻辑会统计两次。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 110, MOD = 1e6 + 7;

int n, m;
int a[N];
int f[N];
```

```cpp

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> a[i];

    f[0] = 1;
    for(int i = 1; i <= n; i++)
    {
        for(int j = m; j >= 0; j--)
        {
            for(int k = 1; k <= j && k <= a[i]; k++)
            {
                f[j] = (f[j] + f[j - k]) % MOD;
            }
        }
    }

    cout << f[m] << endl;

    return 0;
}
```

3.4 分组背包

3.4.1 通天之分组背包

> **题目来源：** 洛谷
>
> **题目链接：** P1757 通天之分组背包
>
> **难度系数：** ★★

【题目描述】

> 自 01 背包问世之后，小 A 对此深感兴趣。一天，小 A 去远游，却发现他的背包不同于 01 背包，他的物品大致可分为 k 组，每组中的物品相互冲突，现在，他想知道最大的利用价值是多少。

【输入描述】

> 两个数 m,n，表示一共有 n 件物品，总重量为 m。
>
> 接下来 n 行，每行 3 个数 a_i,b_i,c_i，表示物品的重量，利用价值，所属组数。
>
> $0 \le m \le 1000$，$1 \le n \le 1000$，$1 \le k \le 100$，a_i,b_i,c_i 在 int 范围内。

【输出描述】

> 一个数，最大的利用价值。

【示例一】

> 输入：
>
> 45 3
>
> 10 10 1
>
> 10 5 1
>
> 50 400 2
>
> 输出：
>
> 10

【解法】

跟之前的分析方式基本一致，相信你们自己就能把它做出来。

因为一个组里面最多只能挑一个元素，所以我们就以一个组为单位。

1. 状态表示：

   $dp[i][j]$ 表示从前 i 组中挑选物品，总重量不超过 j 的情况下，最大的价值。

   那么 $dp[n][m]$ 就是最终结果。

2. 状态转移方程：

   根据第 i 组选什么物品，可以分若干情况讨论。设选择的物品重量为 a ，价值为 b ，此时的最大价值就是 $dp[i-1][j-a]+b$ 。

   因为要的是最大值，所以考虑所有物品之后，取所有情况的最大值就是 $dp[i][j]$ 。

3. 初始化：

   全是 0 即可。

【参考代码】

```
#include <iostream>
#include <vector>

using namespace std;

typedef pair<int, int> PII;

```

```cpp
const int N = 1010;

int n, m, cnt;
vector<PII> g[N]; // g[i]
int f[N];

int main()
{
    cin >> m >> n;
    for(int i = 1; i <= n; i++)
    {
        int a, b, c; cin >> a >> b >> c;
        cnt = max(c, cnt);
        g[c].push_back({a, b});
    }

    // 动态规划
    for(int i = 1; i <= cnt; i++)
    {
        for(int j = m; j >= 0; j--)
        {
            // 在这一组中选择物品
            for(auto& t : g[i])
            {
                int a = t.first, b = t.second;
                if(j >= a) f[j] = max(f[j], f[j - a] + b);
            }
        }
    }

    cout << f[m] << endl;

    return 0;
}
```

3.4.2 排兵布阵

> **题目来源：** 洛谷
>
> **题目链接：** P5322 [BJOI2019] 排兵布阵
>
> **难度系数：** ★★★

【题目描述】

小 C 正在玩一款排兵布阵的游戏。在游戏中有 n 座城堡，每局对战由两名玩家来争夺这些城堡。每名玩家有 m 名士兵，可以向第 i 座城堡派遣 a_i 名士兵去争夺这个城堡，使得总士兵数不超过 m 。

如果一名玩家向第 i 座城堡派遣的士兵数严格大于对手派遣士兵数的两倍，那么这名玩家就占领了这座城堡，获得 i 分。

现在小 C 即将和其他 s 名玩家两两对战，这 s 场对决的派遣士兵方案必须相同。小 C 通过某些途径得知了其他 s 名玩家即将使用的策略，他想知道他应该使用什么策略来最大化自己的总分。

由于答案可能不唯一，你只需要输出小 C 总分的最大值。

【输入描述】

输入第一行包含三个正整数 s, n, m ，分别表示除了小 C 以外的玩家人数、城堡数和每名玩家拥有的士兵数。
接下来 s 行，每行 n 个非负整数，表示一名玩家的策略，其中第 i 个数 a_i 表示这名玩家向第 i 座城堡派遣的士兵数。

对于 100% 的数据：
$1 \leq s \leq 1001 \leq n \leq 1001 \leq m \leq 20000$
对于每名玩家 $a_i \geq 0$，$\sum_{i=1}^{n} a_i \leq m$

【输出描述】

输出一行一个非负整数，表示小 C 获得的最大得分。

【示例一】

输入：

```
1 3 10
2 2 6
```

输出：

```
3
```

【示例二】

输入：

```
2 3 10
2 2 6
0 0 0
```

输出：

```
8
```

【解法】

一个城堡一个城堡分析，对于第 i 个城堡，考虑派遣的人数应该在所有玩家对这个城堡派遣人数中考虑。比如示例二的第三个城堡，我们考虑派遣的人数就是 1 和 13（因为要严格大于两倍，大一点就是最好的）。

因此，把每一个城堡看成一个小组，所有玩家在这个城堡派遣的人数看成一个一个物品，要求的就是在派遣人数不超过 m 的情况下的最大得分，符合分组背包。

小优化：对每个城堡中玩家的派遣人数从小到大排序，这样在选择第 k 个人数的时候，总得分就是 $k \times i$。

1. 状态表示：

   $dp[i][j]$ 表示：分配前 i 个城堡，在总人数不超过 j 的情况下，最大的得分。

   那么 $dp[n][m]$ 就是最终结果。

2. 状态转移方程：

   根据第 i 个城堡分配的人数，分情况讨论。假设分配的是排序后的第 k 个元素，那么分配人数为 $a[i][k]$，此时的最大得分为 $dp[i-1][j-a[i][k]]+i\times k$。

   由于要的是最大值，状态转移方程就是上述所有合法的 k 里面的最大值。

3. 初始化：

   全部为 0 即可。

【参考代码】

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 110, M = 2e4 + 10;

int s, n, m;
int a[N][N];
int f[N][M];

int main()
{
```

```
    cin >> s >> n >> m;
    for(int i = 1; i <= s; i++)
    {
        for(int j = 1; j <= n; j++)
        {
            cin >> a[j][i]; // 翻转一下
            // 小贪心
            a[j][i] = a[j][i] * 2 + 1;
        }
    }

    // 把每一行排序
    for(int i = 1; i <= n; i++)
    {
        sort(a[i] + 1, a[i] + 1 + s);
    }

    // 分组背包
    for(int i = 1; i <= n; i++)
    {
        for(int j = 0; j <= m; j++)
        {
            f[i][j] = f[i - 1][j];
            for(int k = 1; k <= s && a[i][k] <= j; k++)
            {
                f[i][j] = max(f[i][j], f[i - 1][j - a[i][k]] + k * i);
            }
        }
    }

    cout << f[n][m] << endl;

    return 0;
}
```

3.5 混合背包

3.5.1 樱花

> **题目来源：** 洛谷
>
> **题目链接：** P1833 樱花
>
> **难度系数：** ★★

【题目描述】

> 爱与愁大神后院里种了 n 棵樱花树，每棵都有美学值 $C_i(0 \le C_i \le 200)$。爱与愁大神在每天上学前都会来赏花。爱与愁大神是生物学霸，他懂得如何赏花：一种樱花树看一遍过，一种樱花树最多看 $P_i(0 \le P_i \le 100)$ 遍，一种樱花树可以看无数遍。但是看每棵樱花树都有一定的时间 $T_i(0 \le T_i \le 100)$。爱与愁大神离去上学的时间只剩下一小会儿了。求解看哪几棵樱花树能使美学值最高且爱与愁大神能准时（或提早）去上学。

【输入描述】

> 共 $n+1$ 行：
>
> 第 1 行：现在时间 T_s（几时：几分），去上学的时间 T_e（几时：几分），爱与愁大神院子里有几棵樱花树 n。这里的 T_s, T_e， T_e 格式为：`hh:mm`，其中 $0 \le hh \le 23$， $0 \le mm \le 59$，且 hh, mm, n 均为正整数。
>
> 第 2 行到第 $n+1$ 行，每行三个正整数：看完第 i 棵树的耗费时间 T_i，第 i 棵树的美学值 C_i，看第 i 棵树的次数 P_i（$P_i = 0$ 表示无数次，P_i 是其他数字表示最多可看的次数 P_i）。
>
> 100% 数据：$T_e - T_s \le 1000$（即开始时间距离结束时间不超过 1000 分钟），$n \le 10000$。保证 T_e, T_s 为同一天内的时间。

【输出描述】

> 只有一个整数，表示最大美学值。

【示例一】

> 输入：
>
> 6:50 7:00 3
>
> 2 1 0
>
> 3 3 1
>
> 4 5 4
>
> 输出：
>
> 11

【解法】

分类讨论即可。

【参考代码】

```cpp
#include <iostream>

using namespace std;

```

```cpp
const int N = 1e4 + 10, M = 1010;

int n, m;
int t[N], c[N], p[N];
int f[M];

int main()
{
    int t1, t2, t3, t4; char ch;
    cin >> t1 >> ch >> t2 >> t3 >> ch >> t4 >> n;
    m = t3 * 60 + t4 - (t1 * 60 + t2);

    for(int i = 1; i <= n; i++) cin >> t[i] >> c[i] >> p[i];

    for(int i = 1; i <= n; i++)
    {
        if(p[i] == 0) // 完全背包
        {
            for(int j = t[i]; j <= m; j++)
            {
                f[j] = max(f[j], f[j - t[i]] + c[i]);
            }
        }
        else // 多重背包 或 01 背包
        {
            for(int j = m; j >= t[i]; j--)
            {
                for(int k = 1; k <= p[i] && k * t[i] <= j; k++)
                {
                    f[j] = max(f[j], f[j - t[i] * k] + c[i] * k);
                }
            }
        }
    }

    cout << f[m] << endl;

    return 0;
}
```

3.6 多维费用的背包问题

3.6.1 L 国的战斗之间谍